You are the lead AI architect helping me build an **Adaptive Learning Platform for learners with intellectual disabilities and typical learners**.

The goal is simple:

> **Every learner must eventually understand all required concepts in a topic, but the route, pace, difficulty, repetition, hints, and remediation should adapt to the individual learner. Strong learners should progress faster and receive harder/application-based questions; struggling learners should receive simpler explanations, examples, repetition, hints, and prerequisite remediation.**

The existing stack is:
FastAPI + PostgreSQL + SQLAlchemy + React/TypeScript + Groq LLM.

Use the following architecture and rules as the single source of truth.

---

1. CORE PRINCIPLE

Do NOT treat a topic as one score.

Every topic must first be decomposed into **Learning Objectives / Concepts**.

Example:

OSI Model →

- Purpose of OSI
- Physical Layer
- Data Link Layer
- Network Layer
- Transport Layer
- Session Layer
- Presentation Layer
- Application Layer
- Layer identification in scenarios

Each objective must have:

- `objective_id`
- `name`
- `description`

- `importance` (1–3)
- `prerequisites`
- `mastery_threshold`
- `minimum_evidence`
- available question types
- available difficulty levels

The number of learning objectives determines how much assessment a topic requires.

Do NOT use a fixed rule such as "every topic requires 7 questions."

A topic with 3 concepts may require 6 questions; a topic with 9 concepts may require 15+ questions.

The system must ensure **coverage of all required concepts**, not merely achieve a high overall percentage.

---

2. WHAT THE LLM DOES

The LLM is primarily a **content-generation and structuring component**, NOT the real-time adaptive decision maker.

When a teacher uploads a unit:

Step 1 — Extract and structure

Identify:

- Topics
- Subtopics
- Learning objectives
- Prerequisite relationships
- Important concepts
- Examples
- Common misconceptions

Step 2 — Create a learning blueprint

For every objective generate:

- Simple explanation
- Standard explanation

- Worked example
- Simpler example
- Prerequisite explanation
- Hint 1
- Hint 2
- Misconception correction
- Easy questions
- Medium questions
- Hard questions
- Application/scenario questions

Questions must be tagged with:

```
topic_id
objective_id
difficulty
question_type
correct_answer
explanation
hint
misconception
prerequisite_objective
```

The LLM must generate **enough question variants** so the learner does not repeatedly see the same question.

All generated content must be validated and teacher-approved before being served.

The LLM should NOT be called after every student answer.

---

3. QUESTION DIFFICULTY

Use three main levels:

EASY

Recognition, recall, simple identification, direct examples.

MEDIUM

Understanding, comparison, explanation, basic application.

HARD

Application, reasoning, unfamiliar examples, scenario-based questions.

Difficulty should increase gradually:

`EASY → MEDIUM → HARD`

Never jump directly from severe difficulty to a very hard question.

Hard questions are enrichment/challenge questions; they are not required merely to prove basic mastery.

---

4. LEARNER STATE

Maintain mastery separately for every learning objective.

For each objective store:

- `mastery_score`
- `current_difficulty`
- `attempt_count`
- `correct_count`
- `recent_answers`
- `hints_used`
- `consecutive_wrong`
- `independent_correct`
- `status`
- `last_activity`

Do not maintain only one global student score.

The learner's real knowledge state is:

```
Objective 1 mastery
Objective 2 mastery
Objective 3 mastery
```
...

---

5. MASTERY SCORE

Use mastery as an estimate of demonstrated understanding, not simply total correct answers.

A correct answer without help provides strong evidence.

A correct answer after a hint provides weaker evidence.

A wrong answer provides negative evidence.

Recommended evidence:

- Correct independently = 1.0
- Correct after hint = 0.7
- Wrong = 0

Use recent evidence more strongly than very old evidence.

However, NEVER mark an objective mastered from one lucky correct answer.

An objective must satisfy BOTH:

1. `mastery_score >= mastery_threshold` (default 80%)
2. `minimum_evidence` has been collected

The minimum evidence should depend on objective complexity.

Simple factual concept → approximately 2–3 successful demonstrations.

Conceptual/application concept → approximately 3–5 demonstrations.

Complex scenario/application concept → approximately 4–5 demonstrations.

These are configurable values, not hardcoded universal rules.

---

6. TOPIC MASTERY

A topic is NOT mastered merely because its average score is 80%.

Topic mastery requires:

- All required learning objectives have sufficient evidence.
- Every critical objective reaches its mastery threshold.
- Overall weighted mastery reaches the topic threshold.
- No important prerequisite gap remains.

Use:

```
weighted_topic_mastery = Σ(objective_mastery × objective_weight) / Σ(objective_weight)
```

Default topic mastery threshold = 80%.

But a learner must still pass every required/critical objective.

Example:

OSI:

Physical = 90
Data Link = 91
Network = 88
Transport = 92
Session = 42
Presentation = 90
Application = 89

Even if overall mastery is >80%, the topic is **NOT mastered** because Session Layer is weak.

The engine must return to Session Layer.

---

7. BASELINE ASSESSMENT

Before teaching a topic, conduct a short baseline assessment.

The baseline should sample the important learning objectives.

Its purpose is NOT to determine whether the learner passes.

Its purpose is to estimate:

- What the learner already knows
- Which objectives are weak
- Which difficulty level is appropriate

- Which prerequisites may be missing

Initialize the learner's concept profile from baseline performance.

A learner with high baseline mastery should skip unnecessary basic instruction and move toward medium/hard/application questions.

A learner with low baseline mastery should begin with simple explanations and easy questions.

---

8. LEARNING LOOP

After baseline:

```
Select objective → Teach → Question → Grade → Update mastery → Decide next action
```

After EVERY answer, the learning engine must make four decisions:

1. **Which objective should be addressed next?**
2. **What difficulty should be used?**
3. **What instructional support is required?**
4. **Has the objective/topic been mastered?**

---

9. CORRECT ANSWER RULES

Correct + low mastery

Increase difficulty slightly or provide another question at the same level.

Example:

```
Easy → Medium
```

Do not immediately jump to Hard.

Correct + medium mastery

Continue at Medium or introduce a slightly harder question.

Correct + high mastery

Move toward Hard/application questions.

If the learner successfully demonstrates the objective sufficiently, mark it mastered.

Strong learners should therefore progress faster.

---

10. WRONG ANSWER RULES

First wrong answer

Do NOT immediately assume the learner does not understand.

Show:

- Correct answer
- Short explanation
- Simpler example

Then provide an easier question testing the SAME objective.

```
Wrong → Support → Easier practice
```

Second consecutive wrong answer

Increase instructional support:

```
Wrong → simpler explanation → worked example → easier question
```

If the learner continues failing, investigate prerequisites.

Repeated wrong answers

Look for a repeated misconception or prerequisite gap.

Do NOT keep asking essentially the same question.

Instead:

```
Repeated error → misconception/prerequisite remediation → easy
diagnostic question
```

---

11. HINT-DEPENDENT CORRECT ANSWERS

If the learner requests a hint and then answers correctly:

Count it as partial evidence.

Do NOT treat it the same as an independent correct answer.

If the learner repeatedly needs hints:

Status = `HINT_DEPENDENT`

Keep the learner at the current difficulty and provide another guided example until they can answer independently.

---

12. PREREQUISITE FAILURE

Every learning objective can have prerequisites.

Example:

```
Transport Layer → prerequisite: Network Layer
```

If the learner repeatedly fails Transport Layer questions:

Check the prerequisite.

If Network Layer mastery is insufficient:

```
Transport → Network remediation → diagnostic → return to Transport
```

Do not endlessly simplify the current concept if the learner is missing the underlying prerequisite.

---

13. STRUGGLING LEARNER

If the learner repeatedly fails even Easy questions:

```
Easy → simpler explanation → example → hint → easier diagnostic
```

If the learner reaches approximately **5 consecutive failures at the easiest level**:

```
Freeze learning journey → notify teacher → teacher intervention
```

The learner should never be trapped in an infinite failure loop.

The teacher can:

- Add support
- Review the learner
- Reset the activity
- Assign prerequisite material
- Unblock the learner

---

14. STRONG LEARNER

If the learner repeatedly answers correctly:

Do NOT make them answer many repetitive easy questions.

Instead:

```
Easy → Medium → Hard → Application
```

Once sufficient evidence is obtained, mark the objective mastered and move to the next unmastered objective.

The goal is **time-to-mastery**, not number of questions answered.

---

15. OBJECTIVE SELECTION

When choosing the next activity, prioritize:

1. Critical unmastered objectives
2. Prerequisite gaps
3. Weakest objectives
4. Objectives currently being learned
5. Objectives requiring additional evidence
6. Enrichment for already-mastered objectives

Do not randomly select questions.

The engine should know WHY it selected every activity.

Example:

```
NEXT_ACTIVITY_REASON = PREREQUISITE_GAP
```

or:

```
NEXT_ACTIVITY_REASON = LOW_MASTERY
```

or:

```
NEXT_ACTIVITY_REASON = READY_FOR_CHALLENGE
```

or:

```
NEXT_ACTIVITY_REASON = EVIDENCE_REQUIRED
```

---

16. WHEN DOES A CONCEPT END?

An objective ends when:

```
mastery_score >= threshold
```
AND
```
minimum evidence satisfied
```
AND
```
independent performance demonstrated
```
AND
```
no unresolved critical misconception exists
```

Then:

```
objective.status = MASTERED
```

The learner moves to the next required objective.

---

17. WHEN DOES A TOPIC END?

A topic ends only when:

- Every required objective is mastered.
- Every critical objective is mastered.
- Required evidence exists for each objective.
- Weighted topic mastery ≥ threshold.
- No unresolved prerequisite gap exists.

Then:

```
topic.status = MASTERED
```

The learner can proceed to the next topic.

---

18. END-OF-TOPIC ASSESSMENT

After all objectives appear mastered, give a short **post-test/integration assessment** using new questions.

The questions should test whether the learner can apply the concepts without the exact examples used during practice.

Compare:

```
Baseline → Final/Post-test
```

Calculate:

```
Improvement = Post-test score - Baseline score
```

Store this for teacher analytics.

---

19. IMPORTANT RULE: DON'T CONFUSE LEARNING WITH DIFFICULTY

The adaptive engine is NOT simply:

```
Correct → harder
Wrong → easier
```

It is:

```
Performance → diagnose knowledge state → choose objective → choose difficulty → choose support → reassess
```

A learner may receive a HARD question while another learner receives an EASY explanation about the same topic.

Both are following the same learning objectives but through different paths.

---

20. ACCESSIBILITY FOR INTELLECTUAL DISABILITIES

The system should adapt not only difficulty but also **instructional support**.

Support levels:

Level 0

Normal explanation + question.

Level 1

Simpler language.

Level 2

Worked example / visual explanation.

Level 3

Hint.

Level 4

Prerequisite recap + guided practice.

Level 5

Teacher intervention.

The learner should never see technical terms such as:

```
mastery score
adaptive routing
low mastery
escalation rule
```

Instead show friendly language:

"Let's try an easier example."

"Let's look at this idea again."

"Here's a hint."

"Let's go back to an earlier idea."

---

21. IMPORTANT ARCHITECTURE SEPARATION

LLM

Responsible for:

- Content generation
- Learning objective extraction
- Question generation
- Explanations
- Examples
- Hints

- Misconceptions
- Prerequisites
- Question tagging

Learning Engine

Responsible for:

- Learner state
- Mastery calculation
- Objective selection
- Difficulty selection
- Support selection
- Prerequisite navigation
- Topic completion
- Escalation
- Post-test triggering

The LLM must NOT make the final adaptive decision during learning.

The adaptive engine must be deterministic and explainable.

---

22. REQUIRED OUTPUT FROM THE SYSTEM

For every `next_activity`, the backend should internally be able to explain:

student_id
topic_id
objective_id
activity_id
difficulty
support_level
reason
current_mastery
consecutive_wrong
hint_dependency

Example:

objective: Transport Layer
difficulty: EASY
support_level: 2
reason: LOW_MASTERY
mastery: 42
consecutive_wrong: 2

The student only sees the learning activity, not these technical values.

---

23. IMPLEMENTATION PRIORITY

Build in this order:

1. Learning Objective model
2. Question/content model
3. Student concept mastery model
4. Baseline
5. Answer grading
6. Mastery update
7. Adaptive decision engine
8. Prerequisite navigation
9. Topic mastery gate
10. Teacher escalation
11. Post-test
12. Analytics

Do NOT start with sophisticated ML.

A deterministic, explainable adaptive engine is sufficient for the first version and is easier to validate scientifically.

FINAL DESIGN PRINCIPLE

The system must optimize for:

> **Every learner reaches the same learning objectives, but not necessarily through the same path.**

A strong learner may need:

```
5 questions → mastery → next concept
```

A struggling learner may need:

```
explanation → easy question → hint → example → prerequisite → practice
→ medium question → mastery
```

Both learners eventually reach:

```
ALL REQUIRED OBJECTIVES MASTERED → TOPIC MASTERED
```

Design the database, APIs, adaptive engine, prompts, algorithms, and frontend around this principle.

Before writing implementation code, first produce:

1. The complete data model.
2. The exact mastery calculation.
3. The adaptive decision tree/state machine.
4. The question-generation JSON schema.
5. The `next_activity` algorithm/pseudocode.
6. Three complete simulations: a strong learner, an average learner, and a struggling learner progressing through the same topic.

Do not add features that are unrelated to this core learning loop.